为了帮助 Coding Agent 高效完成 AntennaPod 等大型项目的迁移，我为你设计并归档了这套 **"ArkTS-Native Migration Engine"** Skills。这套技能体系严格遵循 Anthropic Skills 格式，旨在解决 Java 到 ArkTS 迁移中的底层范式不匹配问题。

📂 ArkTS-Native 迁移技能库 (Skills Archive)

1. Skill: Concurrency-Actor-Refactor (并发范式重构)

- **Description**: 将 Java 基于共享内存和锁的并发模型（synchronized/volatile）转换为 ArkTS 的 Actor 模型和异步锁机制。
- **Rules**:
  - 禁止将 Java 的 synchronized 翻译为简单的变量访问。
  - 所有跨线程共享的对象必须标记为 @Sendable。
  - 使用 ArkTSUtils.locks.AsyncLock 替换 synchronized 块以避免死锁。
  - 状态变更必须是异步的（async/await）。
- **Examples**:
  - **Input (Java)**: public synchronized void add() { count++; }
  - **Output (ArkTS)**:
    TypeScript
    ```
    @Sendable export class Counter {
      private count: number = 0;
      private lock: ArkTSUtils.locks.AsyncLock = new ArkTSUtils.locks.AsyncLock();
      async add() { await this.lock.lockAsync(() => { this.count++; }); }
    }
    ```
- **Resources**:(https://developer.huawei.com/consumer/cn/doc/harmonyos-guides-V13/arkts-async-lock-introduction-V13)

2. Skill: Static-Metadata-Binding (反射静态化绑定)

- **Description**: 解决 ArkTS 禁止运行时动态修改对象形状及反射受限的问题。
- **Rules**:
  - 检测到 Class.forName() 或 Method.invoke() 时，强制生成静态映射表（Static Dispatch Map）。
  - 禁止使用 ESObject 绕过检查，必须定义显式接口（Interface）。
  - 对于依赖注入（如 Hilt），将其重构为基于单例工厂的静态注入模式。
- **Examples**:
  - **Input (Java)**: method.invoke(obj, args);
  - **Output (ArkTS)**:
    TypeScript
    ```
    // 建立静态分发器
    const actionMap: Record<string, (arg: string) => void> = {
      "play": (url) => player.play(url),
    ```

```
  "stop": () => player.stop()
};
actionMap[methodName](args);
```

- **Resources**:(https://developer.huawei.com/consumer/cn/doc/harmonyos-guides/arkts-restrictions-and-extensions)

3. Skill: Nominal-Type-Integrity (名义类型完整性防护)

- **Description**: 解决 ArkTS 不支持结构化类型（Structural Typing）导致的类型转换错误。
- **Rules**:
  - 即使两个类的属性完全相同，也不允许直接赋值。
  - 在处理 Java 泛型时，必须显式传递类型标识符（Type Discriminator）。
  - 禁止在复杂类型上使用 any 或 unknown。
- **Examples**:
  - **Input (Java)**: List<T> list; if (item instanceof FeedItem)...
  - **Output (ArkTS)**:
    TypeScript
    ```
    @ObservedV2 class FeedItem {
      readonly type: string = "FeedItem"; // 显式类型判别
      @Trace id: number = 0;
    }
    if ((item as FeedItem).type === "FeedItem") {... }
    ```

- **Resources**:(https://developer.huawei.com/consumer/cn/doc/harmonyos-guides/arkts-v1-v2-migration)

4. Skill: Reactive-State-UI-Binding (响应式状态绑定)

- **Description**: 将 Android 的命令式 UI 更新（runOnUiThread/Handler）转换为 ArkUI 的声明式状态管理。
- **Rules**:
  - 禁止在异步回调中直接操作 UI 组件引用。
  - 使用 getUIContext() 获取当前运行上下文，并绑定到 @State 或 @Trace 变量。
  - 后台任务（TaskPool/Worker）完成后，必须通过状态同步机制更新 UI。
- **Examples**:
  - **Input (Java)**: runOnUiThread(() -> textView.setText(data));
  - **Output (ArkTS)**:
    TypeScript
    ```
    @Component struct Player {
      @State message: string = "";
      build() { Text(this.message) }
      onDataLoad(data: string) {
        this.getUIContext().runInGuiThread(() => { this.message = data; });
```

```
  }
}
```

- **Resources**:(https://dev.to/zhongxiang/in-depth-understanding-of-ui-context-in-harmonyos-arkts-4mk6)

5. Skill: Android-To-Harmony-Kit-Mapper (系统服务能力映射)

- **Description**: 将 Android 核心系统组件（Intent, Service, Room）映射到鸿蒙对应的 Kit 。

- **Rules**:
  - Intent -> Want (显式) 或 startAbilityByType (隐式) 。
  - Room/SQLite -> relationalStore (RDB)，且必须手动执行 resultSet.close() 。
  - BroadcastReceiver -> CommonEventManager 。
  - ExoPlayer/Media3 -> AVPlayer (Media Kit)，且必须申请 AUDIO_PLAYBACK 长时任务 。

- **Mapping Table**:
  - **Storage**: @Entity -> RdbPredicates; DAO -> RdbStore.insert/query.
  - **Event**: EventBus -> emitter 或 commonEventManager.
- **Resources**:(https://developer.huawei.com/consumer/cn/doc/harmonyos-guides/media-kit-V5)

🛠 如何在 Coding Agent 中部署

将以下指令加入 Agent 的 **System Prompt**：

Agent Capabilities Integration

1. 启用: 遇到 Java 锁机制时，自动应用 @Sendable 和 AsyncLock 模式。
2. 启用: 遇到反射逻辑时，自动生成接口分发映射表。
3. 启用: 严格禁止非名义类型转换，自动添加类型判别因子。
4. 启用: 将所有 UI 更新转换为基于 @Trace 的状态驱动模式。
5. 启用: 根据预设映射表，将 Android 核心 API 替换为鸿蒙 Kit API。

迁移原则

- 性能优先：优先使用 TaskPool 处理并发。
- 内存安全：所有 RdbStore 结果集必须显式 close() 。

- 规范先行：严格遵守 ArkTS 语法子集，禁止使用 var/any。

这套 Skills 能够显著降低 LLM 在迁移 AntennaPod 这种 10w+ 行代码项目时的“幻觉”率，确保生成的代码不仅语法正确，且符合鸿蒙底层的运行逻辑。